



VINUNIVERSITY

Lab 02: Expressions, Variables, Operators, Assignments, Statements, and Objects in Java.

Week 01, Mar 06 2026

**College of Engineering and Computer Science,
Object-Oriented Programming & Data Structures Spring**

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your code and functions/modules should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables and functions should have meaningful names, and code should be organized into function-s/methods where appropriate.
- Academic honesty is required in all work you submit to be graded. You should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions (modified to provide your student ID, your name, etc).
- Codeforces platform: <https://vinuni26oop.contest.codeforces.com/>
- You should **upload your .java file(s) to Codeforces before the end of the laboratory session** unless the instructor gave a specified deadline. **No Canvas or Codeforces submission** will get **0** point.
- Submit a separate .java file for each lab problem, with the exact filename specified in each lab question. For example: **Lab1_P1_V202401744.java**.
- Late submission of lab practice without an approved extension will incur the following penalties:
 1. No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
 2. The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
 3. The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.

Problem 1: Coordinate Geometry

Write a program that takes two floating-point numbers representing (x, y) coordinates and determines which quadrant the point lies in.

The Cartesian plane is divided into four quadrants based on the signs of the x and y coordinates:

- Quadrant 1: Both $x > 0$ and $y > 0$ (Upper Right)
- Quadrant 2: $x < 0$ and $y > 0$ (Upper Left)
- Quadrant 3: Both $x < 0$ and $y < 0$ (Lower Left)
- Quadrant 4: $x > 0$ and $y < 0$ (Lower Right)
- On Axis/Origin: If $x = 0$ or $y = 0$.

Input

Two doubles (x and y) separated by a space.

Output

The quadrant name (e.g., "Quadrant 1") or "On Axis/Origin". If a non-numeric input is provided, your error handling block must catch it and print exactly: "Error: X and Y coordinates must be numbers."

Note

You must NOT use any "if" or "else" statements. Utilize inline conditional expressions to handle the logic. Furthermore, you must implement an exception handling block to safely manage non-numeric inputs.

Examples

Test Case 1:

Input:

1.5 -2.0

Output:

Quadrant 4

Test Case 2:

Input:

abc 5.5

Output:

Error: X and Y coordinates must be numbers.

Problem 2: Academic Rating

Implement a system that converts a numeric GPA integer into a letter grade.

Detailed Logic: The program maps the 4 scale to letter grades as follows:

- 4: Letter grade A (Excellent)
- 3: Letter grade B (Good)
- 2: Letter grade C (Satisfactory)
- 1: Letter grade D (Poor)
- 0: Letter grade F (Fail)
- Other: Any other integer is "Invalid".

Input

An integer between 0 and 4.

Output

A single character (A, B, C, D, F). If the user inputs an integer outside the 0-4 range, or if an exception is caught due to text input, print exactly: "Invalid".

Note

You must NOT use any "if" or "else" statements. Use a modern Java branching expression to solve this problem. Wrap your input reading in an exception handling block to handle users entering text instead of an integer.

Examples

Test Case 1:

Input:

4

Output:

A

Test Case 2:

Input:

text

Output:

Invalid

Problem 3: Selective Summation

Calculate the sum of all integers from 1 up to a user-defined limit N . However, only include numbers that are multiples of 3 or 5 in the final sum.

Input

A positive integer N .

Output

The resulting integer sum. If the user provides a number less than or equal to zero, print exactly: "Error: Limit must be greater than 0." If an exception occurs during input, print exactly: "Error: Limit must be an integer."

Note

You must NOT use any "if" or "else" statements inside your loop. Use an inline mathematical expression combined with a conditional expression to achieve the selective summation. Enclose your logic in a robust error-handling structure.

Examples

Test Case 1:

Input:

10

Output:

33

Test Case 2:

Input:

-5

Output:

Error: Limit must be greater than 0.

Problem 4: Dynamic Mean Calculation

Continuously read a series of non-negative integers from the user. The process should terminate when a negative integer is entered. Calculate the mathematical average of all non-negative numbers provided.

Input

A stream of integers ending in a negative value.

Output

The average formatted to two decimal places. If the first valid entry is negative, print exactly: "No data". If a non-integer is encountered and caught by your error block, print exactly: "Error: Input stream interrupted by non-integer data."

Note

You must NOT use any "if" or "else" statements. Manage your program's flow entirely through the condition check of your loop and conditional expressions. Use proper exception handling to terminate gracefully if bad data is entered.

Examples

Test Case 1:

Input:

5 15 10 -1

Output:

10.00

Test Case 2:

Input:

5 hello

Output:

Error: Input stream interrupted by non-integer data.

Problem 5: Prime Sequence Generator (Optional)

Generate a sequence of the first N prime numbers. A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself.

Input

A positive integer N .

Output

A space-separated list of the first N primes. If $N \leq 0$, print exactly: "Error: Number of primes must be positive." If an exception is thrown due to bad input, print exactly: "Error: Input must be a valid integer."

Note

You must NOT use any "if" or "else" statements. Use boolean variables, loop conditions, and conditional expressions to maintain state and control what gets printed. Employ an exception handling block to secure the input phase.

Examples

Test Case 1:

Input:

3

Output:

2 3 5

Test Case 2:

Input:

0

Output:

Error: Number of primes must be positive.